CMDBUF Studio 时钟与复位启动时序

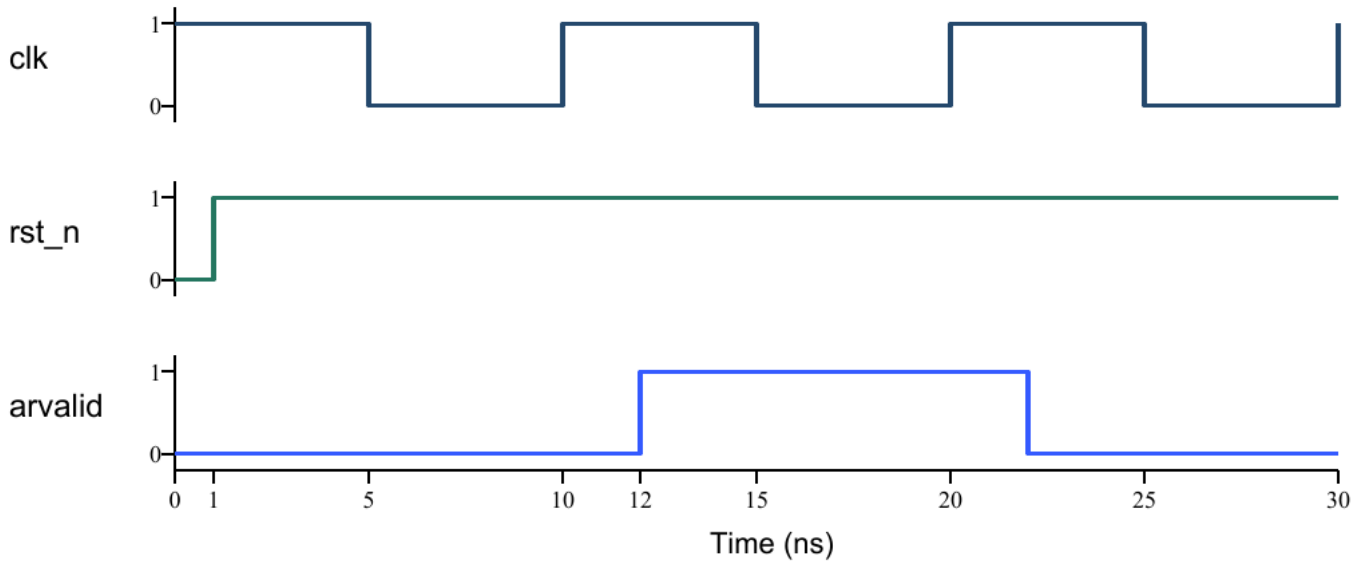
原生 Tk 本机版操作补充 2026 年 11 月 11 日

当前 SV 激励环境从仿真开始运行时钟，clk 初始为 1。低有效 rst_n 在 t=0 拉低，于 #1 释放，之后一直保持 1。Testbench 的时间单位为 1 ns、精度为 1 ps，所以这里的 #1 就是 1 ns。

默认时序

时间	clk / rst_n	行为
0 ns	1 / 0	时钟初始高电平，复位有效。
1 ns	1 / 1	释放复位，此后 rst_n 一直为 1。
5 ns	0 / 1	默认时钟第一次下降。
10 ns	1 / 1	首个正常上升沿，启动 main_phase 的 cycle=0。
12 ns	1 / 1	经过模板的两段 #1，驱动 cycle=0 输入。
20 ns	1 / 1	DUT 采样 cycle=0 的输入；本拍之后准备下一组。

下图来自 VCS 实际生成的 FSDB，经 Verdi 转换后提取跳变。首拍正例在 cycle=0 拉高 arvalid，12 ns 完成驱动，并在 20 ns 接受请求。



实测首拍用例 main_first_cycle seed 17

用例和 GUI 中如何使用

在“用例编辑”中继续只修改 main_phase。所有随项目提供的 TC 已改为 rst_n=1；不需要自行编写启动复位。环境在运行 main_phase 之前已建立复位时序，时钟低高时间和低电平暂停仍可通过原有 clk_*_ns 字段设置。

```
for (int unsigned cycle=0; cycle<num_cycles; cycle++) begin
    @(posedge clk);
    #1;
    begin_cycle(cycle);
    rst_n = 1;
    rready = 3'b111;
    clk_low_ns = 5; clk_high_ns = 5; clk_pause_ns = 0;
    drive_control();
    #1;
    sample_context();
    // Keep the template AR / Bank / MEM if blocks here.
    drive_inputs();
end
```

原先的 rst_n=(cycle>=3) 会在运行中再次请求拉低复位，现应改为 rst_n=1。环境会用 MAIN_RESET_FIXED_HIGH_AFTER_1NS 明确拒绝重新拉低的请求，即使关闭断言 fatal 也一样；不会静默改写用户赋值。tc_reset 名称保留用于计划兼容，现检查启动复位和停钟后保留事务，不再中途复位取消事务。

查看记录和重放

“结果与波形”的运行记录增加 clock_reset.csv，保存实际时钟与复位跳变，时间单位为 ps。输入审计会检查初值、1 ns 释放、每个上升沿和下降沿，以及整个运行期间复位保持高电平。关闭波形生成时仍保留该记录。

```
make run CASE=tc_directed SEED=17 WAVES=1
make wave REGRESSION=latest CASE=tc_directed SEED=17
make tc-test JOBS=2
make replay REGRESSION=latest CASE=tc_directed SEED=17
```

新归档重放会额外比较 clock_reset.csv 的精确内容。旧归档使用当时保存的仿真二进制和时序，不会被新启动策略改写。说明 38 覆盖说明 32 等旧文档中的多周期复位和中途再次复位示例；历史显式波形回放仍按其文件内容执行。

实际验证

本次启动验收 47 项检查通过，包含 10 个 FSDB 的时钟复位逐跳变核对、首拍有效请求、非法再次复位、9 次通用回归、新旧归档重放，以及 NPI 从 1 ns 复位释放后提取首个 10 ns 工作沿。main_phase 验收为 11 个正例和 13 个预期失败反例；检查模式与 fatal 专项 76 项检查也全部通过。

复位采用当前低有效异步复位接口。工具按要求保留 1 ns 脉冲，不会为了其他 RTL 的同步复位需求自行插入时钟脉冲或延长复位。NPI 已验证生成结果，本次未重新执行 Formal 证明。